

REFACTORTACTICS

Mappa tattica 3D e pathfinding

FRTCellId, grafo multilivello, A*, LOS e revisioni

Documento: PDR-06

Versione: 0.1 - Consolidato iniziale

Data: 3 agosto 2026

Baseline tecnica: Unreal Engine 5.8

Specifica dati e servizi spaziali autorevoli.

Controllo del documento

Campo	Valore
Stato	Bozza consolidata per revisione tecnica
Versione	0.1 - Consolidato iniziale
Data	3 agosto 2026
Autore	RefactorTactics / documentazione consolidata con supporto AI
Regola di prevalenza	Decisioni esplicite del progetto > requisiti consolidati > proposte PDR > ricerca web di supporto

Sommario esecutivo

La mappa è un grafo tattico 3D compatto. FRTCellId usa X, Y e Layer; celle e archi contengono stato logico, mentre rendering e collisione sono aggregati. A* è autorevole, NavMesh è supporto visivo.

Decisioni consolidate
Costi interi/fixed-point, servizi separati per path/LOS/targeting e cache legata alla revisione del grafo.

Assunzioni operative
Il modello di serializzazione compatta e la tassonomia completa dei Layer vengono bloccati dopo il graybox 2D e il primo ponte.

Indice del documento

1. Modello spaziale
2. FRTCellId
3. Cell data
4. Graph transitions
5. Graybox generator
6. A* autorevole
7. Cost model
8. Cache e revisioni
9. LOS/targeting
10. Multilivello roadmap
11. Test e debug

1. Modello spaziale

La mappa è un grafo tattico 3D discreto. Un nodo rappresenta una posizione valida; un arco rappresenta una transizione consentita. X/Y descrivono il piano logico, Layer distingue posizioni sovrapposte come terreno, ponte, tetto e tunnel.

2. FRTCellId

```
USTRUCT(BlueprintType)
struct FRTCellId
{
    GENERATED_BODY()

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 X = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int32 Y = 0;

    UPROPERTY(EditAnywhere, BlueprintReadWrite)
    int16 Layer = 0;

    bool IsValid() const;
    friend bool operator==(const FRTCellId&, const FRTCellId&) = default;
};
```

Per usarlo come chiave di TMap serve GetTypeHash stabile. Per serializzazione di rete si può aggiungere NetSerialize solo quando i range sono fissati; durante Fondazioni la serializzazione standard riduce rischio.

3. Cell data

Categoria	Campi suggeriti
Identità	CellId, chunk ID, local index, revision
Geometria logica	elevation, world anchor, surface type, normal/facing hints
Movimento	base cost, blocked flags, unit profile restrictions
Combattimento	directional cover, opacity, visibility modifiers
Ambiente	water/fire/electric state, hazard intensity/duration
Runtime	occupant StableUnitId, interactions, enabled tags

4. Graph transitions

Gli archi sono dati di prima classe: una porta, un ponte o un ascensore modifica una transizione, non solo una mesh. Ogni arco dichiara From, To, transition type, physical cost, requirements, enabled state e revision. Le transizioni speciali possono produrre eventi nella resolution.

5. Graybox generator

- 1 Input: width, height, cell size, origin, default Layer=0.
- 2 Generare array compatto di celle e adiacenze 4-way iniziali.
- 3 Creare una sola istanza renderer per chunk o InstancedStaticMesh, non Actor per cella.
- 4 Line trace/select converte hit world -> candidate cell -> lookup valido.
- 5 Debug overlay mostra X,Y,Layer, cost, blockers e graph revision.
- 6 Salvare la mappa come Data Asset o dati generati riproducibili per il test.

6. A* autorevole

UE 5.8 espone FGraphAStar in AIModule/GraphAStar.h. Il progetto fornisce un graph adapter e un query filter. Il graph adapter espone validità e vicini; il filter decide costo, euristica e traversabilità.

```
using FRTGraphSearch = FGraphAStar<FRTGraphAdapter>;

FRTGraphAdapter Graph(MapState);
FRTPathQueryFilter Filter(UnitProfile, MoveBudget, MapState);
FRTGraphSearch Search(Graph);
EGraphAStarResult Result = Search.FindPath(Start, Goal, Filter, OutPath);
```

Nota API

Lo snippet è architetturale e va adattato alla firma/template della patch UE 5.8 bloccata. L'API ufficiale richiede TGraph e TQueryFilter con metodi specifici; verificare la compilazione nel branch Fondazioni.

7. Cost model

Componente	Tipo	Uso
PhysicalCost	Intero/fixed-point	Distanza, salita, superficie, transizione.
TacticalPreference	Intero, solo preview/bot	Cover, esposizione, formazione.
RiskCost	Intero, conoscenza consentita	Hazard pubblico e intenti alleati.
Validity	Boolean/reason	Blocco, profilo unità, budget, porta.

Il path autorevole per il commit deve usare regole competitive; la UI può offrire varianti di preferenza senza cambiare quali archi siano legalmente percorribili.

8. Cache e revisioni

Chiave cache proposta: GraphRevision + UnitMovementProfileId + Start + Goal + Budget + QueryPolicyId. Una modifica a porta, ponte, hazard bloccante o costo incrementa la revisione del chunk e una revisione aggregata. La cache non deve restituire path costruiti su un grafo precedente.

9. LOS e targeting

Pathfinding, line of sight, targeting e trajectory sono servizi distinti. Un arco percorribile non implica visibilità; una cella visibile non implica target valido. Separare i servizi evita di codificare eccezioni delle abilità dentro A*.

Servizio	Input	Output
Path	start, goal, unit profile, budget, map rev	path, cost, reason
LOS	source cell/height, target cell/height, opacity	visible, blockers, confidence
Targeting	ability definition, source, candidate, state	valid, reason, affected cells
Trajectory	source, direction/arc, blockers	segments, impact, obstruction

10. Multilivello roadmap

- Fondazioni: Layer 0, griglia 2D, ostacoli e A* locale.
- Passo 2: archi speciali tra Layer 0 e 1 per scale/rampe/ponte.
- Passo 3: tunnel sovrapposti e selezione livello camera/UI.
- Passo 4: porte/ponte runtime con revisioni e invalidazione cache.
- Passo 5: ascensori e transizioni temporizzate integrate nella resolution.

11. Test e debug

Suite minima: equality/hash e conversione world-grid di FRTCellId; A* su mappa vuota, ostacolo, no path, budget e costi superficie; tie-break stabile con adiacenze permutate; invalidazione cache su GraphRevision; debug draw di nodi, archi, costi e path; profiling CSV/Insights con nodi espansi, cache hit e microsecondi per query.

Fonti tecniche verificate: Epic Games, FGraphAStar C++ API, UE 5.8.